Espressioni, lvalue e operatori

Marco Cerbella

Lezione 11 – 19 ottobre 2026

1 Espressioni

Definizione. Un'*espressione* è una sequenza di costanti (literal), identificatori e operatori che il programma valuta eseguendo le operazioni indicate. Lo scopo di un'espressione in un programma può essere ottenere il valore risultante, produrre *effetti collaterali* della valutazione, oppure entrambi.

Una singola costante, un literal stringa o l'identificatore di un oggetto o di una funzione è già di per sé un'espressione.

1.1 Effetti collaterali

Un'espressione ha un *effetto collaterale* (*side effect*) se modifica uno stato al di fuori del proprio ambiente locale, cioè se ha un'interazione osservabile con il mondo esterno oltre a restituire un valore.

```c
if (a == 5) {              // restituisce un valore
    printf("Hello World!\n");
}
if (a = 5) {               // restituisce un valore E ha un effetto collaterale
    su a
    printf("Hello World!\n");
}
```

Osservazione. Il secondo `if` è *sbagliato* come modo di verificare una condizione: `a = 5` assegna 5 ad `a` e vale 5, quindi la condizione è sempre vera.

1.2 Tipo di un'espressione

Ogni espressione ha un tipo: è il tipo del valore che risulta dalla sua valutazione. Se l'espressione non produce alcun valore, ha tipo `void`. Per esempio, se `a` ha tipo `int`, anche `a + 1` ha tipo `int`.

Per capire come vengono valutate le espressioni in C servono quattro concetti fondamentali: *precedenza* e *associatività* degli operatori, *sequence point* e *lvalue/rvalue*.

2 Lvalue e rvalue

Definizione. Un *lvalue* è un'espressione che designa un oggetto. Per *oggetto* si intende una locazione di memoria il cui contenuto può rappresentare valori.

L'esempio più semplice di lvalue è il nome di una variabile. La «l» iniziale significava originariamente *left*: poiché un lvalue designa un oggetto, può comparire a sinistra di un operatore di assegnamento (ma anche a destra), come in

```
espressioneSinistra = espressioneDestra
```

Le espressioni che rappresentano un valore senza designare un oggetto si chiamano, per analogia, *rvalue*. Un rvalue può comparire a destra di un operatore di assegnamento, ma non a sinistra.

Esempio.

Espressione	Categoria	Uso
`7`	rvalue	`7 = a` è un errore; `a = 7` è corretto
`a`	lvalue	`a = 7`
`a+1`	rvalue	`a + 1 = 8` è un errore; `a = a + 1` è corretto
`'c'`	rvalue	`'c' = a` è un errore; `a = 'c'` è corretto
`a+b`	rvalue	`a + b = 11` è un errore; `a = a + b` è corretto

2.1 Gli lvalue possono stare anche a destra di `=`

In `a = 7; b = a;`

1. quando una variabile è a sinistra di `=`, ne *modifichiamo* il valore con quello dell'espressione a destra;
2. quando una variabile è a destra di `=`, ne prendiamo il valore memorizzato.

2.2 Variabili

Le variabili sono lvalue: sono contenitori di valori, qualunque cosa il cui valore possa cambiare in memoria. Per esempio, con `int a = 7;` e `int b = 256;` (little endian, 4 byte ciascuna) la variabile `a` occupa i byte a partire dall'indirizzo 2700 con contenuto `07 00 00 00`, e `b` quelli a partire da 2704 con contenuto `00 01 00 00`.

2.3 Costanti e lvalue

Un oggetto può essere dichiarato `const`. In tal caso, anche se è un lvalue, non si può usare a sinistra di un assegnamento.

```
1 int a = 1;
2 const int b = 2;
3 b = 20;      // Errore: b è dichiarata const int
```

L'operando sinistro di un assegnamento deve essere un *lvalue modificabile*.

Definizione. Un *lvalue modificabile* è un lvalue che non è dichiarato con il qualificatore `const`.

Si usano le costanti quando si vuole essere sicuri che il valore non cambi durante l'esecuzione del programma. Un valore va assegnato subito, nella definizione, perché dopo non è più modificabile:

```
1 const double pi;          // sbagliato: nessun valore iniziale
2 const double pi = 3.14;   // corretto
3 pi = 3.14;                // sbagliato: pi non è modificabile
```

3 Precedenza e associatività degli operatori

3.1 Precedenza

Un'espressione può contenere più operatori. In questo caso la *precedenza* degli operatori determina quale parte dell'espressione è l'operando di ciascun operatore. Come nelle regole usuali dell'aritmetica, gli operatori `*`, `/` e `%` hanno precedenza maggiore di `+` e `-`. Per esempio

`a - b * c` equivale a `a - (b * c)`.

Se si vogliono raggruppare gli operandi in modo diverso si usano le parentesi: `(a - b) * c`.

3.2 Associatività

Se due operatori in un'espressione hanno la stessa precedenza, la loro *associatività* stabilisce se vengono raggruppati con gli operandi da sinistra a destra o da destra a sinistra. Gli operatori aritmetici associano da sinistra a destra, gli operatori di assegnamento da destra a sinistra:

- `a / b / c` equivale a `(a / b) / c`;
- `a = b = c` equivale a `a = (b = c)`.

3.3 Tabella di precedenza e associatività

Prec.	Operatori	Associatività
1	Postfissi: `[]` `()` `.` `->` `++` `--` (e `(tipo){lista}`)	da sinistra a destra
2	Unari: `++` `--` `!` `~` `+` `-` `*` `&` `sizeof`	da destra a sinistra
3	Cast: `(tipo)`	da destra a sinistra
4	Moltiplicativi: `*` `/` `%`	da sinistra a destra
5	Additivi: `+` `-`	da sinistra a destra
6	Shift: `<<` `>>`	da sinistra a destra
7	Relazionali: `<` `<=` `>` `>=`	da sinistra a destra
8	Uguaglianza: `==` `!=`	da sinistra a destra
9	AND bit a bit: `&`	da sinistra a destra
10	XOR bit a bit: `^`	da sinistra a destra
11	OR bit a bit: `|`	da sinistra a destra
12	AND logico: `&&`	da sinistra a destra
13	OR logico: `||`	da sinistra a destra
14	Condizionale: `? :`	da destra a sinistra
15	Assegnamento: `= += -= *= /= %= &= ^= |= <<= >>=`	da destra a sinistra
16	Virgola: `,`	da sinistra a destra

Alcuni simboli compaiono due volte nella tabella.

- Gli operatori di incremento e decremento `++` e `--` hanno precedenza maggiore quando sono usati come operatori *postfissi* (come in `x++`) che come operatori *prefissi* (come in `++x`).

- I simboli `+`, `-`, `*` e `&` rappresentano sia operatori *unari*, cioè che lavorano su un solo operando, sia operatori *binari*, che collegano due operandi.

4 Operatori aritmetici

Operatore	Significato	Esempio	Risultato
`*`	moltiplicazione	`x * y`	prodotto di x e y
`/`	divisione	`x / y`	quoziente di x diviso y
`%`	modulo	`x % y`	resto di x diviso y
`+`	addizione	`x + y`	somma di x e y
`-`	sottrazione	`x - y`	differenza di x e y
`+` (unario)	segno positivo	`+x`	il valore di x
`-` (unario)	segno negativo	`-x`	l'opposto aritmetico di x

Gli operandi degli operatori aritmetici sottostanno alle regole seguenti:

- solo l'operatore `%` richiede operandi *interi*; gli operandi di tutti gli altri operatori possono avere qualunque tipo aritmetico;
- gli operandi sono soggetti alle usuali conversioni aritmetiche (lezione 10). Il risultato di una divisione tra due operandi interi è anch'esso un intero.

Esempio. Con `short n = -5;`

Espressione	Conversione implicita	Tipo	Valore
`-n`	promozione intera	`int`	5
`n * -2L`	n promosso a `long` (la costante `-2L` è `long`)	`long`	10
`8 / n`	promozione intera	`int`	-1
`8 % n`	promozione intera	`int`	3
`8.0 / n`	n convertito a `double` (perché 8.0 è `double`)	`double`	$-1{,}6$
`8.0 % n`	errore: `%` richiede operandi interi	—	—

Osservazione. Se entrambi gli operandi di una moltiplicazione o di una divisione hanno lo stesso segno, il risultato è positivo; altrimenti è negativo. Il risultato dell'operazione di modulo, invece, ha sempre lo stesso segno dell'operando sinistro (come si vede sopra: $8\%(-5) = 3$).

5 Operatori di assegnamento

In un'operazione di assegnamento l'operando sinistro deve essere un lvalue modificabile, cioè un'espressione che designa un oggetto il cui valore può essere cambiato. Nell'*assegnamento semplice*, eseguito con l'operatore `=`, il valore dell'operando destro viene memorizzato in questo oggetto.

Esistono anche gli *assegnamenti composti*, che combinano nello stesso passo un'operazione aritmetica o bit a bit con l'assegnamento:

- `*=` `/=` `%=` `+=` `-=` (operazione aritmetica e assegnamento);
- `<<=` `>>=` `&=` `^=` `|=` (operazione bit a bit e assegnamento).

Nel valutare un assegnamento composto il programma combina i due operandi con l'operazione indicata e assegna il risultato all'operando sinistro: `a += b` equivale a `a = a + b`.

6 Incremento e decremento

Ciascuno dei simboli `++` e `--` rappresenta sia un operatore postfisso sia un operatore prefisso.

Operatore	Effetto collaterale	Valore dell'espressione
`x++` (postfisso)	aumenta x di uno (come `x = x + 1`)	il valore che x aveva *prima* dell'incremento
`++x` (prefisso)	aumenta x di uno	il valore che x ha *dopo* l'incremento
`x--` (postfisso)	diminuisce x di uno (come `x = x - 1`)	il valore che x aveva *prima* del decremento
`--x` (prefisso)	diminuisce x di uno	il valore che x ha *dopo* il decremento

7 Operatori di confronto

Gli operatori di confronto, detti anche *relazionali* e di *uguaglianza*, confrontano due operandi e producono un valore di tipo `int`: 1 se la relazione specificata è vera, 0 se è falsa.

Esempio.

```
int a = 5;
int b = 6;
int c = 0;
c = a < b;      // c vale 1
c = a == b;     // c vale 0
c = b >= a;     // c vale 1
```

Gli operatori di confronto hanno precedenza minore degli operatori aritmetici, ma maggiore degli operatori logici. Inoltre gli operatori di uguaglianza `==` e `!=` hanno precedenza minore degli altri operatori di confronto:

- `a < b && b < c + 1` equivale a `(a < b) && (b < (c + 1))`;
- `a < b != b < c` equivale a `(a < b) != (b < c)`.

8 Operatori logici

Si possono collegare più espressioni con gli operatori logici per formare condizioni composte: le operazioni booleane AND (`&&`), OR (`||`) e NOT (`!`). Come le espressioni di confronto, anche le espressioni logiche hanno tipo `int`: il risultato vale 1 se l'espressione logica è vera e 0 se è falsa.

Qualunque operando con valore 0 viene interpretato come falso; qualunque valore diverso da 0 è trattato come vero. Le condizioni seguenti sono tutte equivalenti:

```
(a < -0.2) || (a > 0.2)
a < -0.2 || a > 0.2
```

```
!(a >= -0.2 && a <= 0.2)
```

L'operatore unario `!` ha precedenza maggiore degli altri.

8.1 Valutazione in corto circuito

Gli operatori `&&` e `||` hanno una particolarità importante: i loro operandi vengono valutati da sinistra a destra e, se il valore dell'operando sinistro è sufficiente a determinare il risultato, l'operando destro *non viene valutato affatto*.

- `&&` valuta l'operando destro solo se quello sinistro produce un valore diverso da zero;
- `||` valuta l'operando destro solo se quello sinistro produce 0.

Esempio.

```
int a = 1;
int b = 0;
int c;
c = a || ++b;     // b non viene incrementato (resta 0), c vale 1
```

```
int a = 1;
int b = 0;
int c;
c = b && ++a;     // a non viene incrementato (resta 1), c vale 0
```